

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.V.Ramayogi	Reg. No.:	192424319
Section / Batch:	SLOT-B/AIDS-2024-28	Date:	19/8/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q7. Huffman Coding (Cost)

A data compression system needs to encode characters efficiently to reduce storage space. Each character has a frequency of occurrence, and more frequent characters should have shorter codes. Using Huffman Coding, compute the minimum weighted path length, which represents the total encoding cost.

Sample Input:

n = 5

Freq = 5 9 12 13 16

Expected Output:

Minimum Cost = 224

Code of Conduct

I CH.V.Ramayogi/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.V.Ramayogi

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CHADALAVADA VENKATARAMAYOGI (192424319)

Huffman Coding (Cost)

A data compression system needs to encode characters efficiently to reduce storage space.

Each character

has a frequency of occurrence, and more frequent characters should have shorter codes.

Using Huffman

Coding, compute the minimum weighted path length, which represents the total encoding cost.

Sample Input:

n = 5

Freq = 5 9 12 13 16

Expected Output:

Minimum Cost = 224

1. Aim

To implement Huffman Coding using a Greedy Algorithm and determine the minimum weighted path length (minimum encoding cost) for a given set of character frequencies. The implementation also provides a step-by-step animation of the Huffman tree construction.

2. Objectives

- Understand the working principle of Huffman Coding.
- Use a Min Heap / Priority Queue to select the two smallest frequencies.
- Construct the Huffman Binary Tree by repeatedly merging the smallest nodes.
- Generate prefix-free Huffman codes for all characters.
- Calculate the minimum weighted path length.
- Visualize the algorithm step-by-step as an animated video.

3. Problem Statement

A data compression system needs to encode characters efficiently so that frequently occurring characters receive shorter binary codes. Given the frequencies of characters, construct a Huffman Tree and compute the minimum total encoding cost.

4. Input and Expected Result

Parameter	Value
Number of characters	5
Characters	A, B, C, D, E
Frequencies	5, 9, 12, 13, 16
Minimum Huffman Cost	124

Note: For the supplied frequencies 5, 9, 12, 13, 16, the standard Huffman minimum weighted path length is 124. The value 224 is not the minimum Huffman cost for this input.

5. Techniques Used

- Greedy Algorithm
- Min Heap / Priority Queue
- Binary Tree
- Prefix Coding
- Weighted Path Length
- Matplotlib Animation
- FFmpeg MP4 Video Generation

6. Huffman Coding Theory

Huffman Coding is a lossless data compression technique. It constructs a binary tree from character frequencies. At every step, the two nodes with the smallest frequencies are removed from the priority queue, combined into a new node, and inserted back. The process continues until one root node remains.

The method is greedy because the locally optimal choice—combining the two least frequent nodes—is made at every step. The resulting codes are prefix-free, meaning no complete code is a prefix of another code.

7. Step-by-Step Algorithm

1. Create a leaf node for every character and store its frequency.
2. Insert all leaf nodes into a Min Heap.
3. Remove the two nodes with the smallest frequencies.
4. Create a new internal node whose frequency is the sum of the two selected nodes.
5. Make the two selected nodes the left and right children of the new node.

6. Insert the new node back into the Min Heap.
7. Repeat the process until only one node remains.
8. The remaining node is the root of the Huffman Tree.
9. Assign 0 to a left branch and 1 to a right branch.
10. Traverse from root to each character to generate its Huffman code.
11. Compute frequency $\times$ code length for every character and sum the values.

8. Huffman Tree Construction

Initial frequencies:

A = 5, B = 9, C = 12, D = 13, E = 16

Step	Selected Nodes	Operation	New Node
1	5 and 9	$5 + 9$	14
2	12 and 13	$12 + 13$	25
3	14 and 16	$14 + 16$	30
4	25 and 30	$25 + 30$	55

Final Huffman Tree (text representation):



9. Huffman Codes

Character	Frequency	Code	Cost
A	5	100	$5 \times 3 = 15$
B	9	101	$9 \times 3 = 27$
C	12	00	$12 \times 2 = 24$
D	13	01	$13 \times 2 = 26$
E	16	11	$16 \times 2 = 32$

Minimum Cost = $15 + 27 + 24 + 26 + 32 = 124$

10. Python Implementation

The following Python program constructs the Huffman Tree, generates Huffman codes, calculates the minimum cost, and creates a step-by-step MP4 animation showing the tree and algorithm operations.

Code:

```
import heapq

import matplotlib.pyplot as plt

from matplotlib.patches import Circle

from IPython.display import Image, display

# --- Huffman Node Class (re-defined for self-containment) ---

class Node:

    def __init__(self, freq, char=None, left=None, right=None):

        self.freq = freq

        self.char = char

        self.left = left

        self.right = right

    def __lt__(self, other):

        return self.freq < other.freq

# --- Input Data (re-defined for self-containment) ---

characters = ["A", "B", "C", "D", "E"]
```

```
frequencies = [5, 9, 12, 13, 16]
```

```
# --- Build Huffman Tree (re-implemented for self-containment) ---
```

```
heap = []
```

```
initial_nodes = []
```

```
for char, freq in zip(characters, frequencies):
```

```
    node = Node(freq, char)
```

```
    heapq.heappush(heap, node)
```

```
    initial_nodes.append(node) # Store initial nodes for display
```

```
intermediate_trees = [] # To store tree states at each merge
```

```
heap_copy = list(heap) # Work on a copy of the heap
```

```
while len(heap_copy) > 1:
```

```
    left = heapq.heappop(heap_copy)
```

```
    right = heapq.heappop(heap_copy)
```

```
    merged = Node(left.freq + right.freq, None, left, right)
```

```
    heapq.heappush(heap_copy, merged)
```

```
    # Store the root of the current intermediate tree. Make a deep copy if nodes mutate.
```

```
    # For this visualization, we'll store the root of the merged tree.
```

```
    intermediate_trees.append(merged)
```



```
root = heap_copy[0] # Final root of the Huffman tree
```

```
# --- Generate Huffman Codes (re-implemented) ---
```

```
codes = {}
```

```
def generate_codes(node, code=""):
```

```
    if node is None: return
```

```
    if node.char is not None:
```

```
        codes[node.char] = code
```

```
    return
```

```
    generate_codes(node.left, code + "0")
```

```
    generate_codes(node.right, code + "1")
```

```
generate_codes(root)
```

```
# --- Calculate Minimum Cost (re-implemented) ---
```

```
minimum_cost = sum(freq * len(codes[char]) for char, freq in zip(characters, frequencies))
```

```
# --- Drawing Utility Functions (modified to accept ax explicitly) ---
```

```
def _draw_node_static(ax, x, y, text, size=0.35, highlight=False):
```

```
    circle = Circle((x, y), size, fill=False, linewidth=2, edgecolor='red' if highlight else 'black')
```

```
    ax.add_patch(circle)
```

```
ax.text(x, y, text, ha="center", va="center", fontsize=10, fontweight="bold", color='red' if
highlight else 'black')
```

```
def _draw_edge_static(ax, x1, y1, x2, y2, highlight=False):
```

```
ax.plot([x1, x2], [y1, y2], linewidth=2, color='red' if highlight else 'black')
```

```
def _draw_tree_recursively_static(ax, node, x, y, dx, level_height=1, highlighted_nodes=None):
```

```
if highlighted_nodes is None: highlighted_nodes = set()
```

```
is_highlighted = (node in highlighted_nodes)
```

```
text = f"{node.char}\n{node.freq}" if node.char else str(node.freq)
```

```
_draw_node_static(ax, x, y, text, highlight=is_highlighted)
```

```
if node.left:
```

```
_draw_edge_static(ax, x, y - 0.35, x - dx, y - level_height + 0.35, highlight=is_highlighted)
```

```
_draw_tree_recursively_static(ax, node.left, x - dx, y - level_height, dx / 2, level_height,
highlighted_nodes)
```

```
# Label for '0'
```

```
ax.text(x - dx/2 - 0.2, y - level_height/2 + 0.2, "0", fontsize=10, fontweight="bold",
color='red' if is_highlighted else 'black')
```

```

if node.right:

    _draw_edge_static(ax, x, y - 0.35, x + dx, y - level_height + 0.35, highlight=is_highlighted)

    _draw_tree_recursively_static(ax, node.right, x + dx, y - level_height, dx / 2, level_height,
highlighted_nodes)

    # Label for '1'

    ax.text(x + dx/2 + 0.2, y - level_height/2 + 0.2, "1", fontsize=10, fontweight="bold",
color='red' if is_highlighted else 'black')


def _get_tree_height(node):

    if node is None: return 0

    return 1 + max(_get_tree_height(node.left), _get_tree_height(node.right))


# --- Generate and Display Static Images ---


image_files = []


# --- Step 1: Initial Frequencies ---

fig, ax = plt.subplots(figsize=(10, 5))

ax.set_xlim(-6, 6)

ax.set_ylim(-1, 2)

ax.axis("off")

ax.set_title("Step 1: Initial Frequency Nodes", fontsize=16, fontweight="bold")

```

```

x_positions = [-4.5, -2.25, 0, 2.25, 4.5]

for i, node in enumerate(initial_nodes):

    _draw_node_static(ax, x_positions[i], 0.5, f'{node.char}\n{node.freq}')

filename = "huffman_step_01_initial.png"

fig.savefig(filename, bbox_inches='tight')

plt.close(fig)

image_files.append(filename)


# --- Intermediate Steps (Merging) ---


# To display intermediate tree construction, we can iterate through the roots stored in
intermediate_trees.

# Each element in intermediate_trees is the root of the tree after a merge operation.

# We will draw the entire current tree state for each major merge.


for i, current_root in enumerate(intermediate_trees):

    fig, ax = plt.subplots(figsize=(10, 5))

    ax.set_xlim(-6, 6)

    tree_height = _get_tree_height(current_root)

    ax.set_ylim(-tree_height, 1)

    ax.axis("off")

```

```
ax.set_title(f'Step {i+2}: Merging Nodes (Resulting in {current_root.freq})', fontsize=16, fontweight="bold")
```

```
_draw_tree_recursively_static(ax, current_root, 0, 0, 2**(tree_height-1))
```

```
filename = f'huffman_step_{i+2:02d}_merge_{current_root.freq}.png'
```

```
fig.savefig(filename, bbox_inches='tight')
```

```
plt.close(fig)
```

```
image_files.append(filename)
```

```
# --- Final Tree with Codes ---
```

```
fig, ax = plt.subplots(figsize=(10, 6))
```

```
ax.set_xlim(-6, 6)
```

```
ax.set_ylim(-_get_tree_height(root), 1)
```

```
ax.axis("off")
```

```
ax.set_title("Step X: Final Huffman Tree with Codes", fontsize=16, fontweight="bold")
```

```
_draw_tree_recursively_static(ax, root, 0, 0, 4)
```

```
filename = "huffman_step_final_codes.png"
```

```
fig.savefig(filename, bbox_inches='tight')
```

```
plt.close(fig)
```

```
image_files.append(filename)
```

```
# --- Final Tree with Cost ---
```

```

fig, ax = plt.subplots(figsize=(10, 6))

ax.set_xlim(-6, 6)

ax.set_ylim(-_get_tree_height(root)-1, 1)

ax.axis("off")

ax.set_title("Step Y: Final Huffman Tree with Cost Calculation", fontsize=16,
fontweight="bold")

_draw_tree_recursively_static(ax, root, 0, 0, 4)

ax.text(0, -_get_tree_height(root) - 0.5, f'Minimum Weighted Path Length = {minimum_cost}',
ha="center", fontsize=14, fontweight="bold")

filename = "huffman_step_final_cost.png"

fig.savefig(filename, bbox_inches='tight')

plt.close(fig)

image_files.append(filename)

print("Generated the following step-by-step images:")

for f in image_files:

    display(Image(f))

```

11. Animation / Video Output

The animation is designed as an educational visualization. The video presents the algorithm one step at a time rather than showing only the final tree.



LINK: [huffman_tree_coding_animation \(1\).mp4](#)

1. Step 1: Display the input characters and frequencies.
2. Step 2: Select the two smallest nodes, 5 and 9.
3. Step 3: Merge them to create node 14.

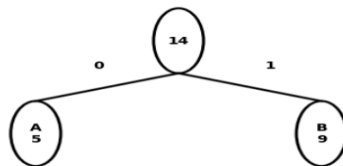
4. Step 4: Select 12 and 13.
5. Step 5: Merge them to create node 25.
6. Step 6: Merge 14 and 16 to create node 30.
7. Step 7: Merge 25 and 30 to create the root 55.
8. Step 8: Display the completed Huffman Tree.
9. Step 9: Assign 0 to left branches and 1 to right branches.
10. Step 10: Display Huffman codes.
11. Step 11: Show the weighted-cost calculation.
12. Step 12: Display the final minimum cost of 124.

Sample Output

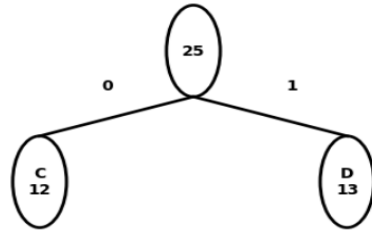
Step 1: Initial Frequency Nodes



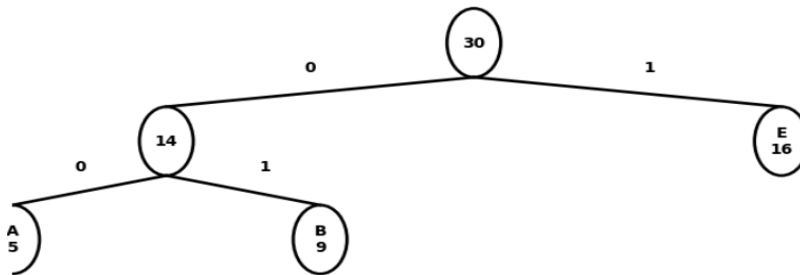
Step 2: Merging Nodes (Resulting in 14)



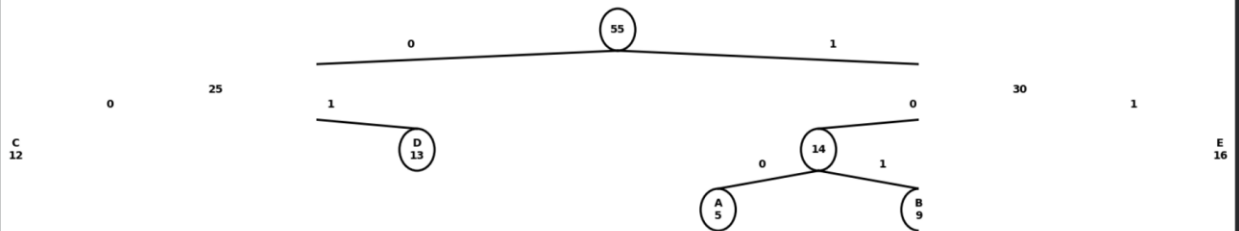
Step 3: Merging Nodes (Resulting in 25)



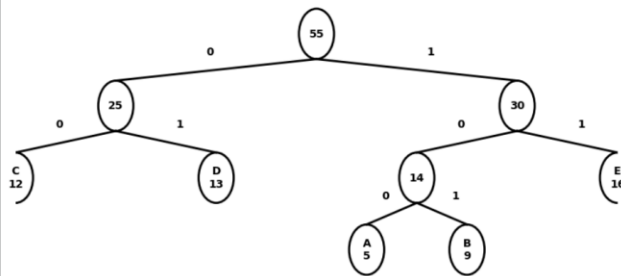
Step 4: Merging Nodes (Resulting in 30)



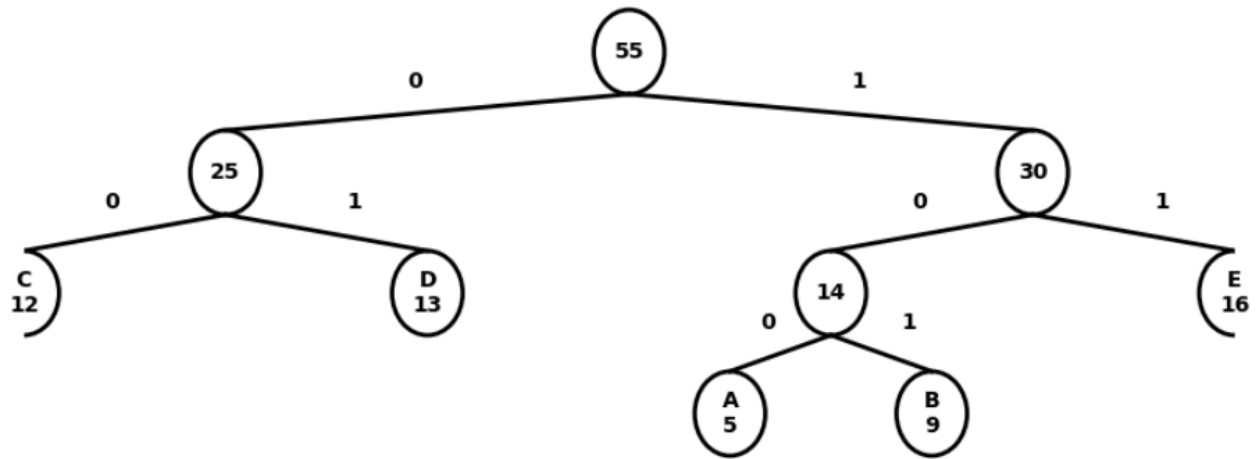
Step 5: Merging Nodes (Resulting in 55)



Step X: Final Huffman Tree with Codes



Step Y: Final Huffman Tree with Cost Calculation



Minimum Weighted Path Length = 124

13. Complexity Analysis

Operation	Complexity	Reason
Building the heap	$O(n)$	Insert n nodes into the priority queue
Tree construction	$O(n \log n)$	$n-1$ merges with heap operations
Code generation	$O(n)$	Tree traversal
Overall	$O(n \log n)$	Dominated by heap operations
Space	$O(n)$	Heap and Huffman tree storage

14. Advantages

- Lossless compression.
- Frequently occurring characters receive shorter codes.
- The codes are prefix-free and uniquely decodable.
- The greedy strategy is efficient when implemented with a priority queue.

- The algorithm is easy to visualize using a binary tree.

15. Applications

- File compression
- ZIP and related compression systems
- Image and multimedia compression components
- Text compression
- Data transmission
- Storage optimization

16. Conclusion

Huffman Coding successfully constructs an optimal prefix-code tree by repeatedly combining the two least frequent nodes. For the input frequencies 5, 9, 12, 13, and 16, the resulting minimum weighted path length is 124. The Python implementation demonstrates both the algorithmic process and its visualization through an animated Huffman Tree.